

NovaTech Enterprise Knowledge Base

Section 1: Company Background

NovaTech Solutions was founded in 2021 as a software engineering and research company. The organization develops enterprise applications, cloud platforms, artificial intelligence systems, and data-processing tools.

During its first year, the company relied heavily on traditional keyword search for internal documents. Employees searched technical manuals, project reports, incident records, and customer-support documents by typing exact words.

The company later discovered that exact keyword matching was not sufficient for many natural-language questions. Different teams often used different words to describe the same technical concept.

For example, backend engineers used the phrase "REST APIs," while product managers often called the same components "backend services." Support engineers sometimes referred to them as "server endpoints."

Because these expressions used different words, traditional keyword search sometimes failed to retrieve the correct documents.

Section 2: Employee Engineering Profile

Aziz Khan joined NovaTech as a backend software engineer. He worked primarily with Python, FastAPI, MySQL, Redis, and REST APIs.

His responsibilities included developing product-management endpoints, implementing authentication, creating authorization rules, integrating payment gateways, and improving database queries.

Aziz also contributed to frontend applications using React and Redux, although his primary responsibility remained backend development.

The e-commerce platform supported more than 10,000 daily users.

Database indexing reduced the average response time of several product queries.

Section 3: General API Performance Improvement

The engineering department investigated several methods for improving API performance.

Some services were slow because database queries scanned large tables. Engineers created indexes for frequently searched columns and optimized SQL queries.

Other services were slow because they repeatedly requested the same information from external systems. The team introduced caching to reduce unnecessary network calls.

These general improvements reduced latency across several internal applications.

However, this section does not describe the special incident involving the payment service.

Section 4: Payment Service Incident INC-7742

On March 14, the payment-processing service experienced severe latency during evening traffic.

The internal incident identifier was INC-7742.

Initial monitoring showed that CPU usage remained normal. Database utilization was also within acceptable limits.

Engineers first suspected MySQL because several payment requests appeared slow. After investigation, they discovered that the database was not the main cause.

The actual problem was repeated calls to an external fraud-detection provider.

Every payment request triggered multiple identical fraud-check requests for the same customer session.

The team solved the incident by storing short-lived fraud-check responses in Redis for ninety seconds.

After the caching change, repeated external requests decreased significantly and payment latency returned to normal.

Section 5: Redis Usage in Shopping Cart

NovaTech also used Redis in the shopping-cart system.

Temporary cart information was stored in Redis so that customers could quickly add or remove products.

Cart records expired automatically after a configured period of inactivity.

This Redis implementation improved shopping-cart responsiveness.

The shopping-cart use case was unrelated to the payment incident described elsewhere in this document.

Section 6: Redis Usage in User Sessions

Another engineering team used Redis for session management.

After successful login, selected session information was temporarily stored for fast access.

This reduced repeated database lookups during authenticated requests.

The session-management implementation improved application performance.

It was not created to solve payment latency or external fraud-detection calls.

Section 7: Redis Usage in Rate Limiting

The API gateway used Redis counters to implement rate limiting.

For each client, the gateway tracked the number of requests received during a configured time window.

When a client exceeded the allowed request count, the gateway temporarily rejected additional requests.

This protected backend systems from excessive traffic.

The rate-limiting design was separate from shopping-cart caching, session management, and payment processing.

Section 8: Authentication Project

The security team implemented authentication using JSON Web Tokens.

When users logged in successfully, the server generated access tokens.

Protected API endpoints validated the tokens before processing requests.

Refresh-token logic allowed clients to obtain new access tokens without forcing users to log in repeatedly.

Authentication answered the question: "Who is the user?"

Section 9: Authorization Project

Authorization was handled separately from authentication.

After a user was authenticated, the application checked roles and permissions.

Administrators could manage products, while ordinary customers could browse products and place orders.

Authorization answered the question: "What is this user allowed to do?"

Although authentication and authorization are related security concepts, they solve different problems.

Section 10: Customer Support Ticket TKT-8821

A customer reported that an order appeared as "processing" for several minutes after payment.

The support ticket identifier was TKT-8821.

The support team initially suspected a payment failure.

Investigation showed that the payment had completed successfully. The delay occurred because an asynchronous order-status worker had temporarily stopped processing messages.

Restarting the worker restored normal status updates.

This issue was not caused by Redis caching, MySQL indexing, or the external fraud provider.

Section 11: Customer Support Ticket TKT-8822

Another customer reported receiving duplicate email notifications after placing an order.

The support ticket identifier was TKT-8822.

Engineers discovered that a retry mechanism submitted the same notification job more than once.

The team added idempotency checks before sending emails.

After the change, duplicate notifications stopped.

This ticket was unrelated to the order-status worker problem.

Section 12: Customer Support Ticket TKT-8823

A third customer reported that the shopping cart became empty after a long period of inactivity.

The ticket identifier was TKT-8823.

Investigation showed that the behavior was expected because temporary cart records had reached their configured expiration time.

The engineering team updated the user interface to explain that inactive carts could expire.

No payment-service failure occurred in this case.

Section 13: Semantic Search Research

The artificial intelligence team investigated semantic search.

Semantic search converts text into embeddings and compares vector similarity.

This allows a query such as "What backend components did Aziz build?" to retrieve a passage containing "Aziz developed REST APIs," even though the exact words are different.

The team used vector databases to store embeddings and retrieve semantically related chunks.

Semantic search performed well for natural-language questions and paraphrased queries.

Section 14: Keyword Search Research

The research team also evaluated keyword retrieval.

Keyword search performed especially well when users entered exact identifiers, technical codes, product names, invoice numbers, or error messages.

For example, a query containing INC-7742 could directly match a document containing the same incident identifier.

Keyword retrieval was less effective when the query and document used different words with similar meanings.

Section 15: Hybrid Search Research

NovaTech combined semantic retrieval with keyword retrieval.

The system executed both retrieval methods and merged their candidate results.

This approach was called Hybrid Search.

Semantic retrieval helped with meaning-based questions.

Keyword retrieval helped with exact terms such as ticket IDs, incident numbers, framework names, and error codes.

Hybrid Search improved candidate recall, but the team discovered another problem: the first retrieved candidates were not always ordered perfectly.

Section 16: Candidate Ranking Problem

During testing, the system retrieved twenty candidate chunks.

Many candidates contained overlapping words.

For example, a question about the payment incident retrieved:

- the actual payment incident report,
- the general API performance section,

- the shopping-cart Redis section,
- the session-management Redis section,
- and the rate-limiting Redis section.

All these passages contained technical words related to performance, services, caching, or Redis.

However, only one passage directly explained why the payment service became slow and how the problem was fixed.

The research team concluded that retrieval and ranking were different problems.

Section 17: Re-ranking Research

To improve final context quality, NovaTech added a cross-encoder re-ranker.

The retrieval system first collected a broad candidate set.

The re-ranker then examined each query-document pair more carefully.

Unlike basic vector similarity, the cross-encoder considered the question and candidate passage together.

The system retrieved twenty candidate chunks and then selected the five most relevant chunks after re-ranking.

This architecture improved the ordering of confusing, similar-looking passages.

Section 18: Machine Learning Model Training

NovaTech's machine learning team trained a forecasting model to estimate future product demand.

Historical sales records, seasonal patterns, promotions, and product categories were used as training features.

The forecasting model helped inventory teams estimate future stock requirements.

This model-training project was separate from embedding generation and semantic search.

Section 19: Embedding Model Usage

The RAG system used an embedding model to convert text into numerical vectors.

Chunks with related meanings were expected to have nearby vector representations.

The embedding model was already trained before NovaTech used it for document indexing.

Uploading a PDF did not retrain the embedding model.

Instead, the existing model generated vectors for document chunks and user queries.

Section 20: Vector Database Storage

Document chunks were converted into embeddings and stored in a vector database.

When a user submitted a question, the same embedding model converted the question into a query vector.

The vector database compared the query vector with stored document vectors.

The most similar candidates were returned to the retrieval pipeline.

The vector database did not generate the final natural-language answer.

Section 21: Large Language Model Role

After retrieval and re-ranking, the selected chunks were placed into the prompt context.

The large language model used that context to generate the final answer.

The language model was responsible for answer generation.

The embedding model was responsible for vector representation.

The vector database was responsible for similarity retrieval.

The re-ranker was responsible for improving candidate order.

These components had different responsibilities.

Section 22: Agriculture Sensor Project

A separate NovaTech research division developed smart irrigation systems.

Sensors measured soil moisture in agricultural fields.

When moisture levels dropped below configured thresholds, the system could recommend irrigation.

Researchers expected the project to reduce unnecessary water consumption.

This project was unrelated to enterprise search and payment processing.

Section 23: Tourism Analytics Project

NovaTech analysts studied winter tourism patterns.

Popular destinations included mountain regions, lakes, and forest areas.

Hotels reported higher occupancy during December and January.

Visitors participating in both mountain and lake activities generally spent more money than visitors who stayed in one location.

This analytics project was unrelated to software incident management.

Section 24: Food Preference Survey

Researchers surveyed visitors about food preferences.

Popular dishes included chicken biryani, fish curry, grilled seafood, and vegetable curry.

Many participants preferred spicy food during winter.

Restaurants used the survey when planning seasonal menus.

Section 25: Final Architecture

NovaTech's production RAG pipeline used multiple stages.

First, documents were divided into chunks.

Second, an embedding model generated vector representations.

Third, dense vectors were stored in a vector database.

A keyword retrieval system maintained information needed for exact-term matching.

When a user asked a question, semantic retrieval and keyword retrieval produced candidate passages.

The results were combined through the hybrid retrieval process.

The system collected a broad set of up to twenty candidate chunks.

A cross-encoder re-ranker evaluated the candidates against the original question.

The five strongest passages were selected.

Those passages were sent to the large language model as context.

The language model then generated the final answer.

The company found that broad retrieval improved recall, while re-ranking improved precision in the final context.